블록체인 상태 구조

개요

CometBFT에서 상태(State)는 블록체인의 현재 상황을 나타내는 핵심 데이터 구조입니다. 상태는 합의 프로세스의 결과로 생성되며, 새로운 블록을 검증하고 생성하는 데 필요한 모든 정보를 포함합니다. CometBFT는 애플리케이션 상태와 합의 상태를 명확히 구분하여 관리합니다.

핵심 개념

상태의 정의

CometBFT에서 상태는 다음과 같은 특성을 가집니다:

- **불변성**: 상태는 직접 변경되지 않으며, 새로운 블록이 적용될 때마다 새로운 상태가 생성됩니다
- **결정성**: 동일한 초기 상태와 동일한 블록 시퀀스는 항상 동일한 최종 상태를 생성합니다
- **검증 가능성**: 상태의 모든 변경사항은 블록체인을 통해 추적하고 검증할 수 있습니다

애플리케이션 상태 vs 합의 상태

애플리케이션 상태 (Application State)

- ABCI 애플리케이션이 관리하는 비즈니스 로직 상태
- 트랜잭션 실행 결과로 변경되는 데이터
- 예: 계정 잔액, 스마트 컨트랙트 상태, 사용자 데이터

합의 상태 (Consensus State)

- CometBFT 합의 엔진이 관리하는 블록체인 메타데이터

- 블록 검증과 합의 과정에 필요한 정보
- 예: 밸리데이터 세트, 합의 파라미터, 블록 헤더 정보

아키텍처

State 구조체

CometBFT의 핵심 State 구조체는 다음과 같은 필드들로 구성됩니다:

```
type State struct {
Version cmtstate.Version

// 불변 필드들
ChainID  string
InitialHeight  int64

// 마지막 블록 정보
LastBlockHeight  int64
LastBlockID types.BlockID
LastBlockTime time.Time

// 밸리데이터 세트
NextValidators *types.ValidatorSet
Validators *types.ValidatorSet
LastValidators *types.ValidatorSet
```

```
LastHeightValidatorsChanged  int64


// 합의 파라미터
ConsensusParams types.ConsensusParams
LastHeightConsensusParamsChanged  int64


// 실행 결과
LastResultsHash []byte
AppHash []byte


// 블록 타이밍
NextBlockDelay time.Duration
}
```

상태 필드 상세 분석

1. 버전 정보 (Version)

```
Version  cmtstate.Version
```

- 블록체인 프로토콜 버전 정보
- 합의 프로토콜과 애플리케이션 프로토콜 버전 포함
- 프로토콜 업그레이드 시 호환성 보장

2. 체인 식별자 (ChainID)

```
ChainID  string
```

- 블록체인 네트워크의 고유 식별자
- 제네시스 블록에서 설정되며 변경되지 않음
- 다른 네트워크와의 구분을 위해 사용

3. 초기 높이 (InitialHeight)

```
InitialHeight  int64
```

- 블록체인이 시작된 높이
- 일반적으로 1이지만, 상태 동기화 시 다른 값 가능
- 제네시스 상태에서 설정

4. 마지막 블록 정보

```
LastBlockHeight  int64

LastBlockID  types.BlockID

LastBlockTime  time.Time
```

- **LastBlockHeight**: 마지막으로 커밋된 블록의 높이
- **LastBlockID**: 마지막 블록의 해시와 파트 세트 해시
- **LastBlockTime**: 마지막 블록의 타임스탬프

5. 밸리데이터 세트 관리

```
NextValidators *types.ValidatorSet
```

```
Validators *types.ValidatorSet

LastValidators *types.ValidatorSet

LastHeightValidatorsChanged  int64
```

- **NextValidators**: 다음 높이에서 사용될 밸리데이터 세트
- **Validators**: 현재 높이의 밸리데이터 세트
- **LastValidators**: 이전 높이의 밸리데이터 세트
- **LastHeightValidatorsChanged**: 밸리데이터 세트가 마지막으로 변경된 높이

6. 합의 파라미터

```
ConsensusParams  types.ConsensusParams

LastHeightConsensusParamsChanged  int64
```

- 블록 크기, 가스 한도, 증거 파라미터 등 합의 관련 설정
- 파라미터 변경이 발생한 마지막 높이 추적

7. 실행 결과

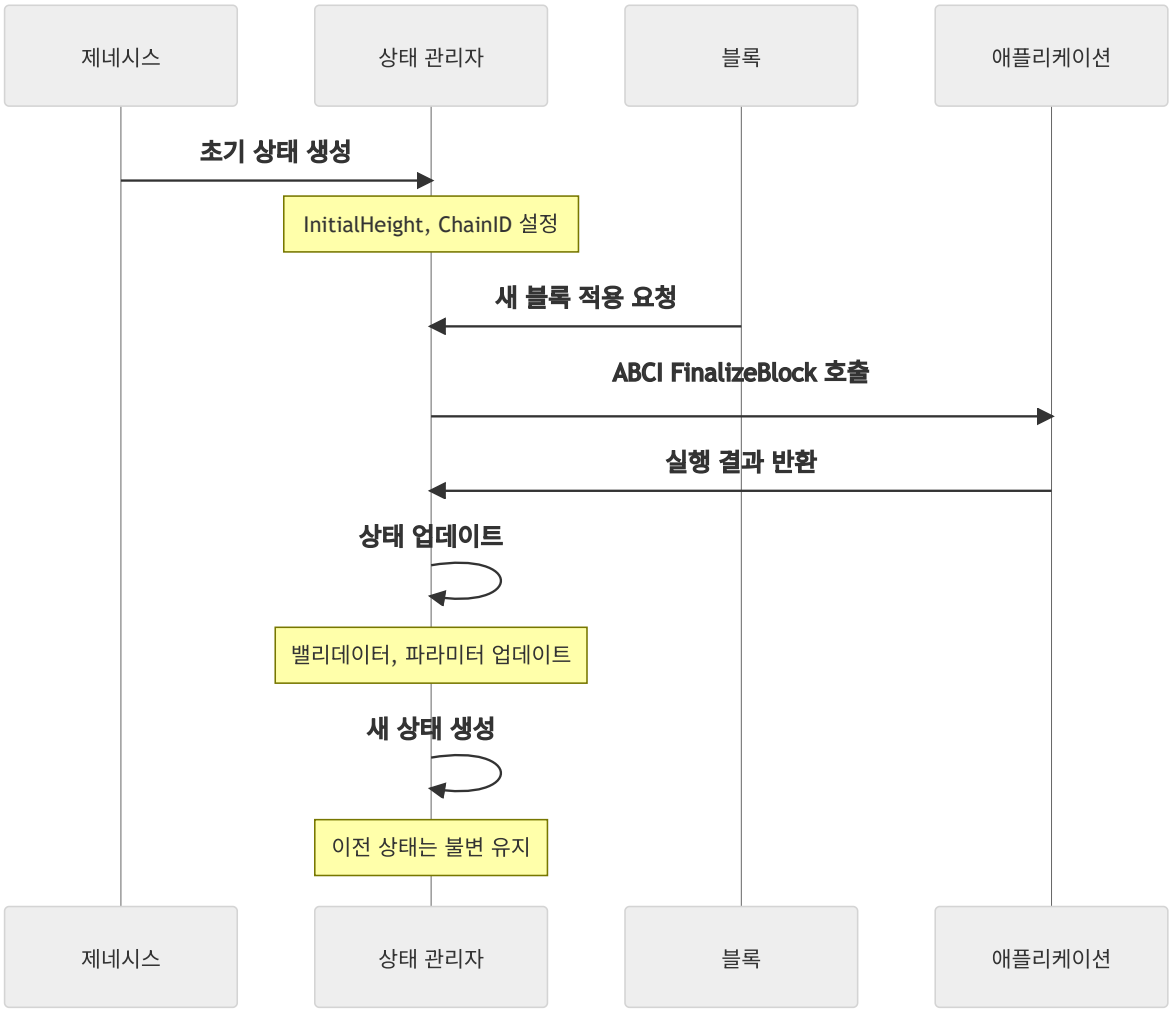
```
LastResultsHash []byte

AppHash []byte
```

- **LastResultsHash**: 이전 블록 실행 결과의 머클 루트
- **AppHash**: 애플리케이션 상태의 해시값

동작 과정

상태 생성과 업데이트



상태 전이 과정

1. **블록 수신**: 새로운 블록이 네트워크에서 수신됨
2. **상태 복사**: 현재 상태를 복사하여 새로운 상태 객체 생성
3. **블록 실행**: ABCI를 통해 애플리케이션에서 블록 실행
4. **상태 업데이트**: 실행 결과를 바탕으로 상태 필드들 업데이트
5. **상태 저장**: 새로운 상태를 데이터베이스에 영구 저장

상태 검증

```go
func validateBlock(state State, block *types.Block) error {
// 블록 높이 검증
if block.Height != state.LastBlockHeight+1 {
return fmt.Errorf("wrong Block.Header.Height")
}
// 이전 블록 해시 검증
if !block.LastBlockID.Equals(state.LastBlockID) {
return fmt.Errorf("wrong Block.Header.LastBlockID")
}
// 밸리데이터 해시 검증
if !bytes.Equal(block.ValidatorsHash, state.Validators.Hash()) {
return fmt.Errorf("wrong Block.Header.ValidatorsHash")
}
return nil
}
```

코드 분석

상태 복사 메서드

```go
func (state State) Copy() State {
```

```
return State{
Version: state.Version,
ChainID: state.ChainID,
InitialHeight: state.InitialHeight,

LastBlockHeight: state.LastBlockHeight,
LastBlockID: state.LastBlockID,
LastBlockTime: state.LastBlockTime,

NextValidators: state.NextValidators.Copy(),
Validators: state.Validators.Copy(),
LastValidators: state.LastValidators.Copy(),
LastHeightValidatorsChanged: state.LastHeightValidatorsChanged,

ConsensusParams: state.ConsensusParams,
LastHeightConsensusParamsChanged: state.LastHeightConsensusParamsChanged,

AppHash: state.AppHash,
LastResultsHash: state.LastResultsHash,
NextBlockDelay: state.NextBlockDelay,
}
}
```

제네시스 상태 생성

```
func MakeGenesisState(genDoc *types.GenesisDoc) (State, error) {
err := genDoc.ValidateAndComplete()
if err != nil {
return State{}, fmt.Errorf("error in genesis doc: %w", err)
}

var validatorSet, nextValidatorSet *types.ValidatorSet
if genDoc.Validators == nil {
validatorSet = types.NewValidatorSet(nil)
nextValidatorSet = types.NewValidatorSet(nil)
} else {
validators := make([]*types.Validator, len(genDoc.Validators))
for i, val := range genDoc.Validators {
validators[i] = types.NewValidator(val.PubKey, val.Power)
}
validatorSet = types.NewValidatorSet(validators)
nextValidatorSet = types.NewValidatorSet(validators).CopyIncrementProposerPriori
}

return State{
Version: InitStateVersion,
```

```
ChainID: genDoc.ChainID,
InitialHeight: genDoc.InitialHeight,


LastBlockHeight: 0,
LastBlockID: types.BlockID{},
LastBlockTime: genDoc.GenesisTime,


NextValidators: nextValidatorSet,
Validators: validatorSet,
LastValidators: types.NewValidatorSet(nil),
LastHeightValidatorsChanged: genDoc.InitialHeight,


ConsensusParams: *genDoc.ConsensusParams,
LastHeightConsensusParamsChanged: genDoc.InitialHeight,


AppHash: genDoc.AppHash,
NextBlockDelay: 0,
}, nil
}
```

설정 및 튜닝

상태 저장 최적화

상태 저장 성능을 최적화하기 위한 설정들:

```
[state]

# 상태 저장 시 동기화 방식

sync = "fsync" # fsync, fdatasync, none


# 상태 압축 사용 여부

compress = true


# 배치 크기 설정

batch_size = 1000
```

메모리 사용량 관리

```
// 상태 캐시 크기 제한

const  maxStateCacheSize = 100


// 오래된 상태 정리

func (store *dbStore) pruneOldStates(retainHeight  int64) error {

// 지정된 높이 이전의 상태들을 정리

return  store.PruneStates(1, retainHeight, retainHeight, 0)

}
```

문제 해결

일반적인 상태 관련 이슈

1. 상태 불일치 오류

```
Error: AppHash mismatch. Expected: ABC123, Got: DEF456
```

해결 방법:

- 애플리케이션 상태와 합의 상태 동기화 확인
- 트랜잭션 실행 로직의 결정성 검증
- 상태 복구 또는 재동기화 수행

2. 밸리데이터 세트 오류

```
Error: validator set hash mismatch
```

해결 방법:

- 밸리데이터 업데이트 로직 확인
- 밸리데이터 세트 변경 시점 검증
- 제네시스 파일의 밸리데이터 정보 확인

3. 상태 저장 실패

```
Error: failed to save state to database
```

해결 방법:

- 데이터베이스 연결 상태 확인

- 디스크 공간 및 권한 확인
- 데이터베이스 백엔드 설정 검토

디버깅 도구

상태 검사 명령어

```
# 현재 상태 정보 조회
cometbft show-state

# 특정 높이의 상태 조회
cometbft show-state --height 12345

# 상태 해시 비교
cometbft compare-states --height1 100 --height2 101
```

로그 분석

```
# 상태 관련 로그 필터링
grep "state" ~/.cometbft/logs/cometbft.log

# 상태 저장 로그 확인
grep "Saved state" ~/.cometbft/logs/cometbft.log
```

참고 자료

- CometBFT State 패키지 문서
- ABCI 애플리케이션 개발 가이드
- 합의 프로세스 상세 분석
- 블록 저장소 구조